FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	*Calvin Josh D. Garcia*
Section:	*AN43*
Professor:	
Date:	

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) *is an intelligent problem-solving approach used in* **Knowledge Representation and Reasoning (KRR)**. *Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.*

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, *including retrieval, reuse, revision, and retention.*
2. **Represent cases effectively** *using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.*
3. **Implement a similarity assessment method** *to identify relevant past cases based on defined attributes.*
4. **Adapt retrieved solutions** *to meet the needs of a new problem scenario based on the chosen case study.*
5. **Demonstrate learning in CBR** *by saving the newly solved case into the system's case base.*
6. **Develop collaboration skills** *through pair programming and joint problem analysis.*
7. **Apply knowledge representation and reasoning concepts** *to real-world problems in disaster response, legal reasoning, or intelligent tutoring.*

Materials Needed

1. Hardware

- *A laptop or desktop computer*
- *Stable internet connection (for research, documentation, submission)*

2. Software

A programming environment:

- *Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)*
- *Any text editor (Notepad++, Sublime, VS Code) if needed*

3. File Formats & Tools

Word processor for documentation:

- *MS Word, Google Docs, or LibreOffice*
- *Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)*

4. Reference Materials

Lecture notes on:

- *Knowledge Representation & Reasoning*
- *Case-Based Reasoning cycle*
- *Similarity measures*

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)

- *Google Drive for shared documentation*
- *GitHub/GitLab for code collaboration*

General Instructions

1. *This* **Technical Assessment (TA3)** *must be completed by Pair.*
2. *Please ensure that you are grouped as pairs in the People tab.*
3. *Select only one case study from the list below.*
4. *Include clear comments in your code to describe its logic.*
5. *Each program should display the correct input, process, and output.*
6. *Ensure that your script displays clear and readable outputs.*
7. *Compile all Python codes and corresponding outputs into one PDF file.*
8. *Save it as* **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. *Submit the* **python file** *and* **PDF** *file.*

📌 *Step-by-Step Instructions*

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*

3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** *stored as dictionaries, JSON objects, arrays, or any structure.*

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- *Attribute matching*
- *Weighted scoring*
- *Simple rule-based comparison*

Output should show: (sample)

✓ *Similarity score or ranking*

✓ *The most relevant past case retrieved*

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** *Adjust evacuation plans based on population size*
- **Legal DSS:** *Modify arguments due to new digital evidence*
- **ITS:** *Give personalized feedback based on student errors*

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ *"New case saved" or similar confirmation*

✓ *Updated case list*

7. Display clear program outputs

Your output must include:

- *New problem description*
- *Retrieved similar case*
- *Adapted final solution*
- *Updated case base*

8. Submission Requirements

Submit the following:

A. Program File

- *Python*

B. Short Documentation (1–2 pages)

Include:

- *Title of chosen case study*
- *Description of your case base*
- *Explanation of similarity method*
- *Adaptation logic*
- *Screenshots of program execution*

C. Pair Reflection (5–7 sentences)

Discuss:

- *What each partner contributed*
- *What challenges you encountered*
- *What you learned about CBR*

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- *Past cases include typhoons, floods, and earthquakes.*
- *New event: Typhoon with expected flooding in urban areas.*
- *System retrieves similar events: Typhoon Ursula (2019) and Typhoon Odette (2021).*
- *It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.*
- *After execution, the system stores new outcomes for improved future reasoning.*

Learning Outcome:

CBR helps governments respond faster by using experiential knowledge rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- *online theft*
- *digital fraud*
- *unauthorized bank transfers*

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks

```
from typing import List, Dict, Any, Tuple


class Case:
    """Represents a single CBR case containing problem features and solution
actions."""
    def __init__(self, case_id: str, case_name: str, problem: Dict[str, Any],
solution: Dict[str, int]):
        self.case_id = case_id
        self.case_name = case_name
        self.problem = problem  # Features: disaster_type, severity_level,
affected_population, infrastructure_damage, accessibility_score
        self.solution = solution  # Actions: rescue_teams, ambulances,
medical_kits, temporary_shelters

    def __repr__(self):
        return f"<Case {self.case_id} ({self.case_name})>"


class CaseBase:
    """Stores historical disaster cases and provides retrieval/retention
mechanisms."""
    def __init__(self):
        self.cases: List[Case] = []

    def add_case(self, case: Case):
        self.cases.append(case)

    def get_all(self) -> List[Case]:
        return self.cases


class CBRSystem:
```

```
    """Implements the complete CBR 4R Cycle: Retrieve, Reuse/Adapt, Revise,
Retain."""

    # Feature weights summing to 1.0
    WEIGHTS = {
        "disaster_type": 0.25,
        "severity_level": 0.25,
        "affected_population": 0.20,
        "infrastructure_damage": 0.15,
        "accessibility_score": 0.15
    }

    # Range normalization for numerical attributes
    MAX_POPULATION = 50000.0  # Upper boundary for population normalization

    def __init__(self, case_base: CaseBase):
        self.case_base = case_base

    # STEP 1: SIMILARITY ASSESSMENT & RETRIEVAL (RETRIEVE)
    def _attribute_similarity(self, key: str, val1: Any, val2: Any) -> float:
        """Calculates normalized local similarity (0.0 to 1.0) for a given
attribute."""
        if key == "disaster_type":
            # Exact match for categorical disaster type
            return 1.0 if str(val1).strip().title() ==
str(val2).strip().title() else 0.0

        elif key in ["severity_level", "infrastructure_damage",
"accessibility_score"]:
            # Distance scoring for 1-5 scales: 1.0 - (|val1 - val2| / 4.0)
            diff = abs(int(val1) - int(val2))
            return max(0.0, 1.0 - (diff / 4.0))

        elif key == "affected_population":
            # Normalized difference based on MAX_POPULATION
            diff = abs(float(val1) - float(val2))
            return max(0.0, 1.0 - (diff / self.MAX_POPULATION))

        return 0.0
```

```
    def calculate_global_similarity(self, target_problem: Dict[str, Any],
historical_problem: Dict[str, Any]) -> float:
        """Computes weighted sum of local attribute similarities."""
        total_score = 0.0
        for key, weight in self.WEIGHTS.items():
            sim = self._attribute_similarity(key, target_problem[key],
historical_problem[key])
            total_score += weight * sim
        return round(total_score, 4)

    def retrieve(self, target_problem: Dict[str, Any], k: int = 2) ->
List[Tuple[Case, float]]:
        """Retrieves top-k most similar cases from the Case Base."""
        scored_cases = []
        for case in self.case_base.get_all():
            sim = self.calculate_global_similarity(target_problem,
case.problem)
            scored_cases.append((case, sim))

        # Sort descending by similarity score
        scored_cases.sort(key=lambda x: x[1], reverse=True)
        return scored_cases[:k]

    # STEP 2 & 3: SOLUTION ADAPTATION & REVISION (REUSE & REVISE)
    def adapt_solution(self, target_problem: Dict[str, Any], nearest_case:
Case) -> Dict[str, int]:
        """
        Rule-based adaptation scaling solution parameters based on
        population ratio and severity level difference.
        """
        hist_prob = nearest_case.problem
        hist_sol = nearest_case.solution

        # Population ratio multiplier
        pop_ratio = float(target_problem["affected_population"]) / max(1.0,
float(hist_prob["affected_population"]))

        # Severity scaling (+20% per severity rank increase)
        sev_diff = int(target_problem["severity_level"]) -
int(hist_prob["severity_level"])
        sev_multiplier = 1.0 + (sev_diff * 0.20)
```

```
        # Combined adaptation factor
        scale_factor = max(0.5, pop_ratio * sev_multiplier)

        adapted_solution = {}
        for resource, count in hist_sol.items():
            adapted_solution[resource] = max(1, round(count * scale_factor))

        return adapted_solution

    def revise(self, solution: Dict[str, int], max_caps: Dict[str, int]) ->
Dict[str, int]:
        """Validates solution against local government inventory capacity
constraints."""
        revised_solution = {}
        for resource, qty in solution.items():
            cap = max_caps.get(resource, qty)
            revised_solution[resource] = min(qty, cap)
        return revised_solution

    # STEP 4: LEARNING & RETENTION (RETAIN)
    def retain(self, new_case_id: str, new_case_name: str, problem: Dict[str,
Any], solution: Dict[str, int]):
        """Stores a newly resolved disaster scenario into the Case Base."""
        new_case = Case(new_case_id, new_case_name, problem, solution)
        self.case_base.add_case(new_case)
        print(f"[RETAIN] Confirmation: New case '{new_case_id}'
({new_case_name}) successfully stored in Case Base.")


if __name__ == "__main__":
    # Initialize Case Base with Historical Philippines Disaster Data
    cb = CaseBase()

    cb.add_case(Case(
        "CASE_001", "Typhoon Ursula (2019)",
        {"disaster_type": "Typhoon", "severity_level": 3,
"affected_population": 8000, "infrastructure_damage": 3, "accessibility_score":
4},
        {"rescue_teams": 8, "ambulances": 12, "medical_kits": 300,
"temporary_shelters": 5}
```

```
    ))

    cb.add_case(Case(
        "CASE_002", "Typhoon Odette (2021)",
        {"disaster_type": "Typhoon", "severity_level": 5,
"affected_population": 25000, "infrastructure_damage": 5,
"accessibility_score": 1},
        {"rescue_teams": 25, "ambulances": 40, "medical_kits": 1200,
"temporary_shelters": 20}
    ))

    cb.add_case(Case(
        "CASE_003", "Luzon Flash Flood",
        {"disaster_type": "Flood", "severity_level": 3, "affected_population":
6000, "infrastructure_damage": 2, "accessibility_score": 3},
        {"rescue_teams": 6, "ambulances": 10, "medical_kits": 250,
"temporary_shelters": 4}
    ))

    cb.add_case(Case(
        "CASE_004", "Mindanao Earthquake",
        {"disaster_type": "Earthquake", "severity_level": 4,
"affected_population": 18000, "infrastructure_damage": 4,
"accessibility_score": 2},
        {"rescue_teams": 18, "ambulances": 30, "medical_kits": 800,
"temporary_shelters": 14}
    ))

    cbr = CBRSystem(cb)

    # Maximum operational capacities of the LGU / Emergency Response Unit
    max_capabilities = {
        "rescue_teams": 30,
        "ambulances": 50,
        "medical_kits": 1500,
        "temporary_shelters": 25
    }

    incident_count = len(cb.get_all()) + 1

    while True:
```

```
        print("  STEP 0: ENTER NEW DISASTER INCIDENT DETAILS")
        try:
            incident_name = input("Incident Identifier/Name (e.g., Typhoon Egay
Urban Flood): ").strip()
            if not incident_name:
                incident_name = f"Incident_{incident_count}"

            d_type = input("Disaster Type (Typhoon / Flood / Earthquake /
Fire): ").strip().title()
            severity = int(input("Severity Level (1 = Minor, 5 = Catastrophic):
"))
            affected_pop = int(input("Estimated Affected Population (e.g.,
15000): "))
            inf_damage = int(input("Infrastructure Damage (1 = Minimal, 5 =
Severe): "))
            access_score = int(input("Accessibility Score (1 = Cut off, 5 =
Accessible): "))

            target_scenario = {
                "disaster_type": d_type,
                "severity_level": severity,
                "affected_population": affected_pop,
                "infrastructure_damage": inf_damage,
                "accessibility_score": access_score
            }

            print("  STEP 1: SIMILARITY ASSESSMENT & CASE RETRIEVAL
(RETRIEVE)")
            retrieved = cbr.retrieve(target_scenario, k=2)

            for rank, (case, sim) in enumerate(retrieved, start=1):
                print(f"  Rank {rank}: {case.case_id} ({case.case_name}) |
Similarity Score: {sim * 100:.2f}%")

            top_case, top_sim = retrieved[0]
            print(f"\n[SELECTED BASE CASE]: {top_case.case_id}
({top_case.case_name}) with {top_sim * 100:.2f}% Match")
            print(f"Historical Problem Features: {top_case.problem}")
            print(f"Historical Recommended Solution: {top_case.solution}")
```

```
            print("  STEP 2 & 3: REUSE, SOLUTION ADAPTATION & REVISION
(REVISE)")
            adapted_sol = cbr.adapt_solution(target_scenario, top_case)
            print("Unconstrained Adapted Solution (Raw Scaling):", adapted_sol)

            revised_sol = cbr.revise(adapted_sol, max_capabilities)
            print("Final Approved Response Plan (Capped to Max Inventory):",
revised_sol)

            print("  STEP 4: LEARNING & CASE RETENTION (RETAIN)")
            new_id = f"CASE_00{incident_count}"
            cbr.retain(new_id, incident_name, target_scenario, revised_sol)
            incident_count += 1

            print("  UPDATED SYSTEM CASE BASE")
            for c in cb.get_all():
                print(f"  - {c.case_id}: {c.case_name} | Type:
{c.problem['disaster_type']} | Severity: {c.problem['severity_level']}")

        except ValueError:
            print("\n[ERROR] Invalid numerical input format. Please enter valid
integers for ratings and population.")

        cont = input("\nWould you like to process another disaster incident?
(y/n): ").strip().lower()
        if cont != 'y':
            print("\nExiting Case-Based Reasoning DSS System. Program
finished.")
            break
```

```
  STEP 0: ENTER NEW DISASTER INCIDENT DETAILS
Incident Identifier/Name (e.g., Typhoon Egay Urban Flood): test
Disaster Type (Typhoon / Flood / Earthquake / Fire): Flood
Severity Level (1 = Minor, 5 = Catastrophic): 5
Estimated Affected Population (e.g., 15000): 20000
Infrastructure Damage (1 = Minimal, 5 = Severe): 5
Accessibility Score (1 = Cut off, 5 = Accessible): 2
  STEP 1: SIMILARITY ASSESSMENT & CASE RETRIEVAL (RETRIEVE)
  Rank 1: CASE_002 (Typhoon Odette (2021)) | Similarity Score: 69.25%
  Rank 2: CASE_003 (Luzon Flash Flood) | Similarity Score: 66.90%

[SELECTED BASE CASE]: CASE_002 (Typhoon Odette (2021)) with 69.25% Match
Historical Problem Features: {'disaster_type': 'Typhoon', 'severity_level': 5, 'affected_population': 25000, 'infrastructure_damage': 5, 'accessibility_score': 1}
Historical Recommended Solution: {'rescue_teams': 25, 'ambulances': 40, 'medical_kits': 1200, 'temporary_shelters': 20}
  STEP 2 & 3: REUSE, SOLUTION ADAPTATION & REVISION (REVISE)
Unconstrained Adapted Solution (Raw Scaling): {'rescue_teams': 20, 'ambulances': 32, 'medical_kits': 960, 'temporary_shelters': 16}
Final Approved Response Plan (Capped to Max Inventory): {'rescue_teams': 20, 'ambulances': 32, 'medical_kits': 960, 'temporary_shelters': 16}
  STEP 4: LEARNING & CASE RETENTION (RETAIN)
[RETAIN] Confirmation: New case 'CASE_005' (test) successfully stored in Case Base.
  UPDATED SYSTEM CASE BASE
  - CASE_001: Typhoon Ursula (2019) | Type: Typhoon | Severity: 3
  - CASE_002: Typhoon Odette (2021) | Type: Typhoon | Severity: 5
  - CASE_003: Luzon Flash Flood | Type: Flood | Severity: 3
  - CASE_004: Mindanao Earthquake | Type: Earthquake | Severity: 4
  - CASE_005: test | Type: Flood | Severity: 5
```